

LLMs e Desenvolvimento de Software — Material-Base do Seminário

2026-09-17 · @Someone · Gerenciamento de Projetos · 20–25 min · 5 apresentadores

Como usar este material

Este é o roteiro mestre do seminário, não o slide. Cada um dos 5 blocos abaixo traz a narrativa, os conceitos de Gerenciamento de Projetos a citar, a demo funcional, as falas-chave e as fontes. Os slides são produzidos a partir daqui.

Decisões já tomadas com o grupo:

- **Demos:** HTML/JS puro, offline, embutidas no slide. Nada depende de internet ou de API na hora da apresentação.
- **Peso da linha do tempo:** ~4 min de história, ~12 min de presente, ~5 min de futuro. A história é gancho narrativo, não aula de história.
- **Camada de GP:** amarrada em cada bloco. Cada apresentador fecha com 1–2 conceitos da aula aplicados ao que acabou de mostrar.

O que ainda falta decidir está marcado como **[DECIDIR]** ao longo do texto.

Uma nota sobre honestidade intelectual, que é o que vai diferenciar este seminário: vários números famosos sobre IA e produtividade **foram revisados ou corrigidos pelos próprios autores**. Citar a versão antiga é o erro mais comum. Onde isso acontece, o texto avisa.

Tese central e arco narrativo

A tese: LLMs não são uma ferramenta de programação a mais — elas derrubam o custo de *produzir* código e, com isso, **deslocam o gargalo do projeto** para requisitos, verificação, integração e governança. O problema deixa de ser tecnológico e vira um problema de gerenciamento de fluxo, qualidade, risco e valor.

A pergunta que organiza os 25 minutos inteiros não é "LLM consegue programar?". É:

Quando a capacidade de gerar software aumenta, qual passa a ser o novo gargalo do projeto?

O arco tem uma abertura provocativa que amarra tudo na disciplina. Na **Aula 1**, o professor afirma: "*Software tem economia de escala na entrega mas não no desenvolvimento.*" O seminário inteiro pode ser lido como um teste dessa premissa — o

LLM é a primeira tecnologia que ataca exatamente ela. Não é contradizer o professor; é mostrar que a premissa foi entendida bem o bastante para ser testada.

Bloco	Era	Pergunta que responde	Conceito de GP dominante
1	1948–2016	De onde veio a ideia, e por que ela falhou três vezes?	Estimativa e gestão de expectativas (Aulas 2 e 5)
2	2017–2022	O que mudou tecnicamente, e por que código foi o primeiro a cair?	Business case e valoração (Aula 2), TAP (Aula 3)
3	2026, parte I	Onde a IA entra no ciclo de vida, de copiloto a agente?	EAP, estimativa, Brooks, Lei dos 90-90 (Aulas 3, 5, 6)
4	2026, parte II	O que a evidência realmente mede?	EVA/SPI/CPI, burndown, QA x QC, Goodhart (Aulas 9 e 10)
5	2027+	Para onde vai, e como se preparar?	Matriz de riscos, apetite/tolerância, governança (Aulas 7 e 10)

Por que esse arco atende aos critérios de avaliação:

- *Profundidade* — o ponto alto é o bloco 4, que coloca lado a lado um experimento com 4.867 desenvolvedores que achou **+26%** e um RCT que achou **-19%**, e explica por que os dois podem estar certos.
- *Conexão com GP* — cada bloco fecha com terminologia exata dos slides, não com referência genérica ao PMBOK.
- *Casos reais* — Georgetown-IBM (1954), Knight Capital, a migração do runtime do GitHub Copilot para Rust (set/2026).
- *Material de apoio* — as demos funcionais são o diferencial visual; o design muda por era.
- *Comunicação e tempo* — 5 blocos de ~4 min, cada um com uma frase de virada ensaiada.

Bloco 1 — 1948–2016: a ideia é velha, o que faltava era escala

Apresentador 1 · ~4 min · **Estética:** terminal verde sobre preto, fonte monoespaçada, ruído de CRT

A mensagem do bloco

A ideia central do LLM — prever o próximo símbolo contando o que veio antes — é de **1948**. O que faltava era dado, hardware e arquitetura. E cada salto dessa era veio acompanhado de um fracasso de gestão de expectativas que se repete hoje quase literalmente.

Narrativa (ordem de fala)

1. **Abertura com o fato-choque.** Em 7 de janeiro de 1954, IBM e Georgetown demonstraram tradução automática russo→inglês com **250 itens de vocabulário e 6 regras gramaticais**. Leon Dostert anunciou que a tradução automática estaria resolvida em *"cinco, talvez três anos"*. Doze anos depois, o relatório **ALPAC (1966)** concluiu: *"não houve tradução automática de texto científico geral, e nenhuma está em perspectiva imediata."* O financiamento americano em tradução automática morreu por ~20 anos.
2. **Volte para 1948.** Shannon gerou texto amostrando bigramas de palavras e obteve: THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT THE CHARACTER OF THIS POINT IS THEREFORE... — coerente por ~4 palavras, incoerente no todo. **É o mesmo modo de falha que hoje chamamos de alucinação.** Em 1951 ele mediu a entropia do inglês em ~1 bit por letra: perplexidade, a régua com que se avalia GPT, é de 1951.
3. **Turing, 1950.** Não pelo teste — pela seção 7: *"Em vez de tentar produzir um programa que simule a mente adulta, por que não produzir um que simule a da criança? (...)* Pouquíssimo mecanismo e muitas folhas em branco." Turing descreveu **pré-treino e fine-tuning em 1950**.
4. **ELIZA, 1966.** Casamento de padrões puro, sem aprendizado. A secretária de Weizenbaum pediu que ele saísse da sala para conversar em privado com o programa. → **roda a demo aqui**.
5. **Os dois invernos.** ALPAC (1966) e o colapso das máquinas LISP em 1987 — um mercado de ~US\$ 500 milhões que sumiu em um ano. E o **Fifth Generation** japonês: 12 anos, escopo congelado no dia zero, derrotado pelo ambiente (CPUs comuns, GUIs, internet nasceram durante o projeto).
6. **A virada estatística e o ramo do código.** Em 1993 a IBM traduziu sem um único linguista, só contando sobre 1,78 milhão de pares de frases do parlamento canadense. E em **2012**, Hindle et al. mediram que **código-fonte é muito mais previsível que inglês**: 2–4 bits de entropia para Java contra ~8 para prosa. O autocomplete de código deles, no Eclipse, já economizava **61%** de teclas. Cinco anos antes do Transformer.

A amarração com Gerenciamento de Projetos

Duas, e as duas são fortes:

(a) Estimativa por extrapolação linear de um piloto — Aula 5. Dostert projetou 100 regras para 20.000 palavras a partir de 6 regras para 250 itens: ~17x mais regras para 80x mais vocabulário. É a falácia de estimativa mais comum em software, e o professor a nomeia: *"uma das maiores causas de fracasso é a subestimação do esforço necessário"*. Some **Hatton (2007)**: pessoas superestimam o tempo de tarefas curtas e subestimam o de tarefas longas.

(b) Desalinhamento estratégico — Aula 2. A frase dos slides: *"Muitas empresas iniciam projetos de software porque 'a tecnologia existe' e não porque há um objetivo de negócio."* O ALPAC foi, na prática, uma **auditoria de projeto**. Ele não matou a tradução automática por ela não funcionar — matou porque a entrega não correspondia à promessa de 1954 e **ninguém recalibrou a expectativa do stakeholder no meio do caminho**. A única defesa contra uma auditoria é ter baselines revisados (Aula 10).

Demo funcional

ELIZA em JavaScript, ao lado de um painel que mostra a regra disparada. O público conversa 30 segundos e percebe que "funciona"; aí você revela que a resposta veio da regra `^(.*)\beu sou (.*)$`. O efeito ELIZA e a sua ruptura, em 60 segundos.

Complemento (se houver tempo): gerador n-grama com slider. O público cola um texto, arrasta o `n` de 1 a 6 e vê o ruído virar linguagem. Com `n` alto e corpus pequeno, o modelo **copia o corpus** — overfitting visível a olho nu, e a resposta pronta para "LLM não é só copiar?". Carregue dois corpora: um texto em português e **um arquivo de código do repositório de vocês**. O código sai visivelmente mais plausível — é Hindle et al. 2012 demonstrado ao vivo.

Falas-chave

"Tudo que vocês vão ver nos próximos 20 minutos começa com uma ideia de 1948: prever o próximo símbolo contando o que veio antes."

"Isso aqui não entende nada. E mesmo assim, em 1966, a secretária de Weizenbaum pediu privacidade para conversar com ele."

Transição para o bloco 2: em setembro de 2016 o Google coloca em produção um tradutor neural com 8 camadas de LSTM e **60% menos erros**. Dez meses depois, oito autores publicam um paper cujo título é uma provocação direta: *Attention Is All You Need*.

Armadilhas — não repetir

Não diga	Diga
"A demo de 1954 traduzia russo automaticamente"	Era um <i>showcase</i> com frases pré-selecionadas e dicionário feito sob medida
"O ALPAC disse que tradução automática é impossível"	Ele redirecionou para tradução assistida ; não proibiu
"Turing criou o teste como definição de inteligência"	Ele considerava a pergunta mal formulada; ofereceu um critério comportamental
"ELIZA passou no Teste de Turing"	Nunca houve teste formal; Weizenbaum argumentou o contrário a vida toda
"O inverno da IA veio porque a IA não funcionava"	O XCON economizou ~US\$ 40 mi para a DEC. Foi falha de expectativa e de mercado
"O Copilot inventou autocompletar código com estatística"	Hindle et al., ICSE 2012 , com plugin funcional no Eclipse

Fontes do bloco

[Turing 1950](#) · [Shannon 1948](#) · [Shannon 1951](#) · [Hutchins — Georgetown-IBM](#) · [Hutchins — ALPAC](#) · [Weizenbaum 1966 — ELIZA](#) · [Hindle et al. 2012 — On the Naturalness of Software](#) · [Brown et al. 1993 — SMT](#)

Bloco 2 — 2017–2022: o Transformer, e por que código caiu primeiro

Apresentador 2 · ~4 min · **Estética: transição do terminal para uma interface web dos anos 2010 — azul, cantos arredondados, flat design**

A mensagem do bloco

2017 não inventou o objetivo de treino — esse é de Shannon. Inventou uma **arquitetura que escala**. E software se revelou o caso de uso mais favorável que existe, por três motivos simultâneos: quase tudo é texto, há bases enormes de código público e — o ponto decisivo — **grande parte do resultado pode ser verificada automaticamente** por compilador, linter e teste.

Narrativa

1. **A provocação do título.** *Attention Is All You Need* (jun/2017) é uma resposta direta ao paper de Bahdanau de 2014, que inventara a atenção como muleta do LSTM. A mensagem: a muleta, sozinha, basta. Sai a recorrência, entra paralelização — e treinar em escala vira viável.
2. **A distância que você acabou de ver encurtar.** Em 2015 um char-RNN de Karpathy treinado em 474 MB do kernel Linux já escrevia C sintaticamente plausível, errando variáveis e dependências longas. Daí até o Copilot não houve uma ideia nova: houve **arquitetura mais escala**.
3. **Três coisas que não são sinônimos** — e esse é o slide conceitual central da apresentação inteira.
4. **A régua que revelou o abismo.** Em 2023 o **SWE-bench** parou de pedir "escreva uma função" e passou a entregar uma *issue* real e um repositório inteiro: 2.294 problemas de 12 repositórios Python. O melhor modelo da avaliação original resolvia **1,96%**. Use esse número como marco histórico, nunca como retrato de hoje — é exatamente o contraste que torna o bloco 3 impressionante.

Conceito	O que é	Exemplo
LLM	O modelo que interpreta e gera texto e código	GPT, Claude, Gemini
Assistente de programação	Produto que liga o modelo ao contexto do dev: IDE, arquivos, repositório	Copilot no VS Code
Agente de software	O modelo dentro de um ciclo planejar → agir → observar → corrigir: lê arquivos, edita, roda testes, itera	Agente que resolve uma issue sozinho

A amarração com Gerenciamento de Projetos

Business case e valoração — Aula 2. A Aula 2 exige uma decisão **"Go / No go"** fundamentada em valoração: ganho financeiro, gasto evitado, ganho de eficiência, ganhos não financeiros. Adotar LLM sem responder *qual desses* se pretende melhorar é literalmente o anti-padrão que o professor marcou nos slides como **"fonte de frustração de engenheiros"**. Proposta concreta para o slide: transformar adoção de IA em **OKR**, não em KPI de vaidade.

Não serve como KR	Serve como KR
% de sugestões do Copilot aceitas	Reduzir o lead time de PR em 20%
Linhas de código geradas por IA	Manter cobertura de testes $\geq 80\%$
Número de prompts por dev	Reduzir defeitos escapados em 15%

Bônus — as três métricas da Aula 5. O professor exige que uma métrica seja **válida, confiável e prática**. "% de código aceito do Copilot" é prática, razoavelmente confiável e **não é válida**: não quantifica o que se quer medir. Critério de triagem limpo, tirado direto dos slides.

Demo funcional

Três painéis em um só slide, todos offline:

1. **Tokenizador ao vivo.** O público digita uma frase em português e um trecho de código; a tela mostra os tokens coloridos e a contagem. Revela na hora por que código é mais "barato" de modelar que prosa.
2. **Mapa de atenção estático.** Uma matriz pré-calculada e embutida como JSON para uma frase curta, com células mais escuras onde o peso é maior. Não precisa rodar modelo nenhum — basta uma matriz salva. Passar o mouse sobre uma palavra acende de onde ela "puxou" contexto.
3. **A ponte com o bloco 1.** Reaproveite o gerador n-grama e coloque, lado a lado, a previsão dele e uma previsão com contexto longo (pré-gravada). A diferença é a atenção, e o público acabou de ver as duas.

Falas-chave

"Código não foi o primeiro a cair por acaso. É texto, existe aos bilhões de linhas na internet, e — diferente de um poema — dá para saber automaticamente se está errado. Compilador, linter, teste."

"Em 2023, o melhor modelo resolvia dois por cento das issues reais. Guardem esse número."

Transição para o bloco 3: guardem também a terceira linha da tabela — *agente*. É nela que 2026 acontece.

Fontes do bloco

[Vaswani et al. 2017 — Attention Is All You Need](#) · [Bahdanau et al. 2014 — atenção](#) · [Karpathy 2015 — char-RNN](#) · [Jimenez et al. — SWE-bench](#)

Bloco 3 — Hoje I: a IA não está só na implementação

Apresentador 3 · ~5 min · Estética: interface contemporânea, escura, com um painel de agente rodando de verdade ao lado

A mensagem do bloco

Quem reduz o tema a "autocomplete de código" perde o ponto. LLMs já atuam em **todo o ciclo de vida** — e a passagem de copiloto para agente não aumenta só a velocidade: ela **move o trabalho humano** de digitar para especificar, delegar, revisar e integrar.

Narrativa

1. **Percorra o ciclo de vida da Aula 1** (Iniciação → Planejamento → Execução → Monitoramento e Controle → Encerramento) mostrando que a IA encosta em todas as fases — mas com **maturidade muito desigual**.
2. **Nomeie as três fases da relação humano-modelo**. Primeira: dev escreve, IA sugere linhas. Segunda: dev pergunta, IA explica e modifica. Terceira, *agentic*: humano dá uma intenção, o agente explora o repo, planeja, altera arquivos, roda testes, observa erros, corrige e apresenta o resultado para revisão.
3. **O caso real, e ele é de 16 de setembro de 2026** — um dia antes desta pesquisa.

Etapa do ciclo	Uso de LLM	A pergunta gerencial
Requisitos	Elicitação, síntese de entrevistas, histórias de usuário, achar ambiguidade	O modelo entendeu a necessidade real do stakeholder?
Planejamento	Decomposição de tarefas, dependências, estimativa auxiliar	O plano representa o trabalho real ou só parece plausível?
Arquitetura	Explorar alternativas, documentar decisões, propor interfaces	Quem responde pelos trade-offs?
Implementação	Geração, refatoração, migração, correção de bug, agentes em issues	Mais código melhora o fluxo ou só aumenta WIP e fila de revisão?

Etapa do ciclo	Uso de LLM	A pergunta gerencial
Testes	Testes unitários, casos extremos, mocks, explicar falhas	Os testes validam o requisito ou só o código gerado?
Code review	Resumo de diff, primeira triagem, sugestões	Como evitar falsa confiança?
Documentação	README, API, explicar legado	Continua sincronizada com a implementação?
Operação	Diagnóstico, análise de log, investigação de incidente	Quanta autonomia um agente pode ter em produção?

O caso GitHub — e ele é uma aula de GP inteira

O GitHub reescreveu o runtime do agente Copilot de TypeScript/Node.js para **mais de 800 mil linhas de Rust em produção**. Agentes de IA escreveram a maior parte do código. O autor estima que o projeto antes exigiria uma equipe por um ou dois anos; foi feito principalmente por **um desenvolvedor em alguns meses**.

Diga em voz alta que isso é um **relato técnico de primeira parte**, não um estudo independente. Isso vale nota.

Mas o ponto não é o volume de código. É **como** foi gerenciado:

O que o GitHub fez	Conceito da disciplina
Recusou o <i>big bang rewrite</i>	Mitigação de risco (Aula 7)
Migrou componente por componente, 128 PRs	Entrega iterativa, pequenos lotes (Aula 3)
Main sempre entregável; 135 releases em ~14,5 semanas	Cadência de entregas (Aula 3)
Rodou os testes E2E existentes a cada etapa; quebrou teste, não entra	Quality gate, Garantia da Qualidade (Aula 9)
Começou por pilotos técnicos: toolchain, lint, CI, build	Redução de incerteza, cone da incerteza (Aula 3)
Lógica pura e bem testada primeiro; subsistemas acoplados por último	Sequenciamento e gestão de dependências (Aula 6)
Rollout progressivo para correlacionar regressão com poucas mudanças	Monitoramento e controle (Aula 10)

A frase a dizer: o sucesso não veio de "a IA escreveu tudo sozinha". Veio de IA dentro de um sistema de engenharia disciplinado.

A amarração com Gerenciamento de Projetos

Três, e as três são de aulas diferentes:

(a) A EAP — Aula 5. O professor é categórico: *"Se algo não está na EAP, não será estimado nem executado."* Se **revisão e validação de código assistido por IA** não é um pacote de trabalho na EAP, esse custo não é estimado nem orçado — e reaparece como estouro. Proposta concreta: incluir esse pacote na EAP do UNESPÃO, dentro do ramo 4.2 (Testes).

(b) A Lei dos 90-90 — Aula 6. Cargill: *"Os primeiros 90% do código representam os primeiros 90% do tempo. Os 10% restantes representam os outros 90% do tempo."* O LLM acelera dramaticamente os primeiros 90% — que já eram a parte barata. Integração, casos extremos e depuração de código que você não escreveu permanecem, ou pioram. **A Lei dos 90-90 prevê que o ganho percebido de IA é sistematicamente superestimado** — e o bloco 4 traz o dado que confirma isso.

(c) A Lei de Brooks — Aula 6. A pergunta mais original possível aqui: **um agente de IA conta como "adicionar pessoa" a um projeto atrasado?** Pelas próprias premissas do professor — tempo de aceleração, custo de comunicação crescente, trabalho especializado — o agente **não paga o custo de aceleração, mas gera custo de comunicação**, na forma de revisão. E a curva PNR: o exercício da Aula 6 mostra que comprimir o prazo em 25% custa **+216% de esforço**. A IA quebra esse limite ou apenas desloca o esforço de escrita para revisão, mantendo a curva intacta? Deixe a pergunta em aberto no slide — é exatamente o tipo de provocação que gera discussão.

Demo funcional

"Agente resolvendo uma issue", reprodução passo a passo. Uma linha do tempo que avança com clique ou automaticamente: lê a issue → busca no repo → propõe um plano → edita 3 arquivos → roda os testes → 2 falham → corrige → testes passam → abre PR. Tudo pré-gravado como JSON e reproduzido em JS puro, com o diff aparecendo de verdade na tela.

O momento pedagógico é o passo em que **dois testes falham e o agente corrige sozinho**. Pare ali e diga: *"isso aqui é o que separa um assistente de um agente — o loop de feedback com uma ferramenta determinística."*

[DECIDIR] Usar uma issue real do repositório de vocês (`00davidсарago00/unespao`) torna a demo muito mais forte: é o projeto da turma, no código da turma. Se rolar, grave a sessão antes e transforme em JSON.

Falas-chave

"Oitocentas mil linhas de Rust. Cento e vinte e oito pull requests. Um desenvolvedor. E a parte importante dessa frase não é nenhum desses números — é que a main nunca deixou de ser entregável."

"Se revisar código de IA não está na EAP, esse trabalho não foi estimado. E ele vai acontecer de qualquer jeito."

Transição para o bloco 4: tudo isso parece ótimo. A pergunta que falta é a única que a disciplina realmente exige: **e a evidência, o que diz?**

Fontes do bloco

[GitHub — Migrating the Copilot runtime to Rust, using Copilot \(16/09/2026\)](#) · [PMI — The Standard for AI in Portfolio, Program and Project Management \(jun/2026\)](#) · revisão sistemática de LLMs em Requirements Engineering (74 estudos, 2025) · TestPilot — geração automática de testes para 1.684 funções de 25 pacotes npm

Bloco 4 — Hoje II: +26%, -19%, e o gargalo que se mudou de lugar
Apresentador 4 · ~5 min · Estética: dashboard, gráficos vivos, fundo claro — contraste deliberado com o bloco 3

Este é o ponto alto da apresentação e o que decide o critério "profundidade da pesquisa". Uma apresentação genérica diria "IA aumenta a produtividade". A literatura séria diz **"depende fortemente do contexto"** — e dá para provar.

O confronto

Coloque os dois estudos lado a lado na tela e deixe o público desconfortável por uns segundos antes de explicar.

	Experimentos corporativos	RCT da METR
Onde	Microsoft, Accenture e uma Fortune 100	Repos open source maduros
Quem	4.867 desenvolvedores	16 mantenedores muito experientes

	Experimentos corporativos	RCT da METR
Familiaridade com o código	Variada	Altíssima, anos no mesmo repo
Métrica	Tarefas concluídas	Tempo de implementação
Resultado	+26,08% de tarefas concluídas	19% mais lentos
Publicação	<i>Management Science</i> , fev/2026	METR, jul/2025

O detalhe mais interessante do estudo da METR: antes das tarefas, os participantes achavam que a IA os deixaria ~24% mais rápidos. **Depois de terem sido 19% mais lentos, ainda achavam que tinham sido ~20% mais rápidos.** Percepção de produtividade e produtividade medida não são a mesma coisa.

A honestidade que ninguém tem — e que vocês vão ter

Quase toda apresentação sobre este tema cita os "19% mais lentos" como prova definitiva de que a IA atrapalha. **Em fevereiro de 2026 a própria METR mudou o desenho do experimento**, por viés de seleção: desenvolvedores passaram a recusar participar de tarefas em que não poderiam usar IA, mesmo a US\$ 50/hora, e **30–50% relataram evitar submeter justamente as tarefas em que a IA mais ajudaria** — *"evito issues em que a IA termina em 2 horas e eu levaria 20"*. A própria METR reconhece que os números **subestimam** o benefício real.

Citar o -19% em 2026 sem citar a correção é desonesto. Fazer isso na frente da turma é o momento mais forte do bloco.

A reconciliação

Não há contradição: as duas perguntas eram diferentes. Um modelo mental melhor que "IA aumenta produtividade em X%":

$$\text{Impacto da IA} = \frac{\text{capacidade do modelo} \times \text{adequação da tarefa} \times \text{contexto disponível} \times \text{qualidade do workflow} \times \text{experiência do usuário}}{\text{custo de verificação}}$$

Isso é síntese analítica, não fórmula validada — diga isso. O ponto: "usar LLM" não é uma variável detalhada o bastante para prever produtividade.

E há um dado do DORA 2026 que confirma isso de forma falsificável: ganho de produtividade de **35–40% em greenfield** contra **~10% ou menos em código legado**. Repare que esse número é *desfavorável ao vendedor* (o DORA é do Google Cloud, que

vende ferramentas de IA) — por isso é o número confiável do relatório. O "ROI médio de 727%" do mesmo relatório é um modelo, não uma medição. **Trate-o como marketing e diga isso.**

O achado que amarra tudo na disciplina

O DORA 2024 mediu algo desconfortável: um aumento de 25% na adoção de IA apareceu associado a melhorias locais — **+7,5%** em qualidade de documentação, **+3,4%** em qualidade de código, **+3,1%** em velocidade de review — e, ao mesmo tempo, a **-1,5%** de throughput de entrega e **-7,2%** de estabilidade.

Em 2025, com 90% de adoção e ~5.000 respondentes, o throughput já havia virado positivo. **A relação negativa com estabilidade permaneceu.**

É possível melhorar a produtividade de uma etapa e piorar o desempenho do sistema.

Desenhe isso no slide:

requisito → design → IMPLEMENTAÇÃO 5× → [gargalo] review → [gargalo] teste → release

O resultado não é 5× mais entregas. É uma fila maior de PRs esperando revisão.

Três fontes com interesses comerciais **opostos** apontam para o mesmo lugar: DORA (Google, vende IA) mede estabilidade caindo; GitClear (vende analytics de qualidade) mede refatoração caindo de 21% para 3,8% e duplicação de blocos subindo 81%; Stack Overflow (neutro, 49 mil devs) mede uso subindo para 84% e **confiança caindo de 43% para 33%**, com 66% frustrados por código "quase certo".

A amarração com Gerenciamento de Projetos

Esta é a seção mais rica da apresentação inteira. Escolham **duas ou três**, não todas.

(a) QA x QC — Aula 9. O professor separa **Garantia da Qualidade** ("fazer certo da primeira vez", foco no processo) de **Controle da Qualidade** ("verificar se o que foi feito está certo", foco no produto). O LLM **inverte a proporção entre os dois**: gerar ficou barato, verificar ficou caro. Tese: **times que adotam LLM sem reforçar o Controle da Qualidade estão otimizando a etapa que já não era o gargalo.**

(b) EVA — Aula 10. $SPI = EV/PV$, $CPI = EV/AC$. O EVA depende do **%AC**, derivado de estimativas históricas. Se a IA muda a produtividade da equipe, **o BAC e o %PC do baseline ficam obsoletos**. Pior: um **CPI > 1 pode ser falso otimismo** — o custo caiu

porque o código saiu rápido, mas a dívida de revisão ainda não entrou no AC. **O EVA subestima sistematicamente o custo de código gerado por IA, porque o retrabalho chega depois do período de medição.**

(c) Burndown — Aula 10. Reutilizem literalmente a pergunta da atividade do professor: *"O que o SPI e o CPI mostram que o burndown, sozinho, não mostra?"* Com IA, a resposta fica óbvia e forte: o burndown despenca nos dias 1-4 e **empaca nos dias 7-10**, quando revisão e integração cobram o preço. O burndown mostra pontos queimados; o CPI mostra que eles custaram caro. É a Lei dos 90-90 com esteroides.

(d) Story points — Aula 5. O professor é enfático: story point *"não pode ser usado como comparação entre times"* e *"não deve ser usado como forma de produtividade"*. O debate público sobre IA (*"aumenta produtividade em X%"*) é **exatamente o uso que ele proíbe**. E mais: ele exige 2-4 sprints de calibração da velocidade — toda adoção de ferramenta de IA **reinicia esse relógio**. A velocidade histórica deixa de ser preditiva no momento em que a capacidade do time muda de degrau.

(e) Lei de Goodhart — Aula 9, último slide. O professor deixou esse slide **sem desenvolvimento**. Preenchê-lo é a jogada de maior retorno do seminário inteiro. "Linhas geradas por IA", "% de sugestões aceitas" e "PRs por dev" são métricas que se destroem ao virar meta: o time aceita sugestões ruins para bater o indicador. Goodhart puro, aplicado a IA, num slide da própria disciplina.

Demo funcional

Simulador de gargalo — a melhor demo da apresentação. Uma esteira animada com quatro estágios: Implementação → Review → Teste → Release, cada um com uma capacidade (itens por dia) e uma fila visível.

O público arrasta **um único slider: a capacidade da Implementação**, de 1× a 5×. E vê, ao vivo:

- o lead time total **quase não cai**;
- a fila antes do Review **explode**;
- o "WIP" total dispara.

Depois vocês mexem no segundo slider — capacidade de Review — e aí sim o lead time cai. **É teoria das restrições demonstrada em 20 segundos, e é a tese inteira do seminário virando animação.**

Mostre também um mostrador de **SPI e CPI** atualizando junto com a simulação: o público vê o SPI subir enquanto o CPI cai. Use as fórmulas do professor na tela.

Complemento: burndown com e sem IA. Dois traçados sobre a mesma sprint de 40 pontos — um linear, outro que despenca e empaca. Use os números da atividade da Aula 10 como base (BAC R\$ 20.000, SPI 0,58, CPI 0,53) para que a turma reconheça o exercício.

Falas-chave

"Um estudo com quatro mil oitocentos e sessenta e sete desenvolvedores achou vinte e seis por cento a mais de tarefas concluídas. Outro, com dezesseis, achou dezenove por cento mais lento. Os dois estão certos."

"E o mais interessante: os desenvolvedores que ficaram 19% mais lentos achavam, depois do experimento, que tinham ficado 20% mais rápidos."

"Mais código não significa mais software entregue."

Transição para o bloco 5: se o gargalo já se mudou uma vez, para onde ele vai na próxima?

Fontes do bloco

[Cui et al. — Management Science, 2026 \(3 experimentos, 4.867 devs\)](#) · [METR — RCT com devs experientes \(jul/2025\)](#) · [METR — mudança de desenho do experimento \(fev/2026\)](#) · [DORA 2025 — State of AI-assisted Software Development](#) · [DORA 2026 — ROI of AI-assisted Software Development](#) · [GitClear — The Maintainability Gap \(2026\)](#) · [Stack Overflow Developer Survey 2025](#)

Bloco 5 — 2027+: a curva, os cenários e como se preparar

Apresentador 5 · ~5 min · Estética: o slide fica quase vazio, só o gráfico e a plateia decidindo — o contraste visual é o argumento

A mensagem do bloco

Provocativo sim, charlatanismo não. A curva mais citada do mundo sobre o futuro da programação **está ficando cega antes de a previsão dela poder ser testada** — e a resposta madura a isso não é apostar num cenário, é escolher a posição que sobrevive nos três.

A curva

A METR mede o **horizonte de tarefa**: a duração de trabalho humano que um modelo consegue completar com 50% de confiabilidade. O paper original (mar/2025) achou que esse horizonte **dobra a cada 207 dias**.

Em janeiro de 2026 a METR republicou os ajustes, e o resultado é o dado mais provocativo do bloco — **a curva não desacelerou, ela acelerou**:

Janela do ajuste	Tempo de duplicação
Série completa, 2019→	196,5 dias (~6,5 meses)
Desde 2023	130,8 dias (~4,3 meses)
Desde 2024	88,6 dias (~2,9 meses)

A progressão medida por modelo: GPT-4 (2023) fazia **3,5 minutos**. GPT-4o (2024), 6 minutos. Claude Opus 4.5 (nov/2025), **320 minutos**. Claude Opus 4.6 (fev/2026), **~14,5 horas**.

E então a METR colocou um aviso na própria página: **"medições acima de 16 h são não confiáveis com nossa suíte atual."**

E agora o contúrdio — porque é isso que separa vocês da apresentação média

A mesma curva tem quatro problemas documentados, três deles admitidos pela própria METR:

- **Tarefas irreais.** Nas 50% de tarefas mais "bagunçadas" — as mais parecidas com trabalho real — **nenhum modelo passava de 30% de sucesso**.
- **Contaminação.** ~34% das tarefas têm solução pública nos dados de treino.
- **Baselines humanos frágeis.** Das 31 tarefas longas (8h+), **apenas 5 têm tempo humano realmente medido**. As outras 26 são estimativas.
- **Intervalos de confiança enormes.** O IC do Opus 4.6 vai de **6 h a 98 h**. O mesmo modelo.

Frase de slide: "A curva mais citada do mundo sobre o futuro da programação tem cinco medições humanas reais na faixa que mais importa."

O mesmo padrão aparece nos benchmarks de código: **SWE-bench Verified saturou em ~95%**, mas quando a Scale AI criou o **SWE-bench Pro** — com repos GPL, para dificultar contaminação — os melhores modelos caíram para ~23%. Um ano depois já estão em

61,5%. Ou seja: o gap de realismo de 47 pontos foi quase todo fechado em 12 meses. **Esse dado é simultaneamente o melhor argumento otimista e a prova de que o problema não era só contaminação.**

A tensão que interessa a quem vai se formar

Dois números oficiais, no mesmo momento histórico:

Fonte	O que diz
BLS (projeção oficial 2024–34)	Desenvolvedores de software: +15,8% , +267.700 vagas
Stanford Digital Economy Lab (ago/2026, folha da ADP)	Trabalhadores de 22–25 anos em ocupações muito expostas a IA: ~19% abaixo do ritmo dos pares menos expostos — e a lacuna vem se ampliando desde ago/2025

O mecanismo medido é **redução de contratação**, não demissão. Trabalhadores experientes não mostram lacuna equivalente.

Os dois podem estar certos: a profissão cresce e a porta de entrada se fecha.

Seja honesto sobre os limites: os próprios autores dizem que **não é estimativa causal**; controlar por escolaridade reduz o efeito quase pela metade; há pré-tendências anteriores ao ChatGPT; e o efeito é específico da amostra ADP. E o detalhe que quase ninguém cita: **especificamente em desenvolvimento de software, a remuneração dos 22–25 cresceu mais rápido que a dos mais velhos no pós-ChatGPT.**

Os cenários

Quatro, e o quarto é o fecho da apresentação.

Cenário	Teria que ser verdade	O que fazer hoje
A. Substituição rápida	A curva se mantiver em ~90–130 dias e generalizar para trabalho bagunçado	Especializar em verificação, especificação e orquestração de agentes
B. Transformação sem substituição (mais sustentado pela evidência)	IA seguir complementar em sistemas reais; gargalo migrar para revisão	Domínio fora de TI + operação de sistemas reais + governança de IA

Cenário	Teria que ser verdade	O que fazer hoje
C. Platô / correção	A saturação refletir limite real, não medida ruim	Fundamentos duráveis: arquitetura, dados, segurança, sistemas distribuídos
D. Bifurcação — A no código verificável, C no resto	Tarefas com oráculo automático se automatizam; tarefas com requisito ambíguo, stakeholder e consequência não	Posicionar-se do lado sem oráculo automático

Calibração externa, para não parecer apocalíptico: o **Metaculus** (1.900 forecasters) projeta AGI em **junho de 2031**; a pesquisa do **NBER** com 69 economistas e 38 superforecasters dá **12,6–14%** de probabilidade ao cenário rápido até 2030. E **Daniel Kokotajlo, autor principal do cenário AI 2027, moveu a própria mediana de 2027–28 para ~2030.**

A amarração com Gerenciamento de Projetos — e este é o fecho

Governança na gramática exata do professor. A Aula 7 define **apetite** ("grau de incerteza que a organização aceita em troca de recompensa"), **tolerância** ("variação aceitável em torno de um objetivo") e **exposição** (probabilidade × gravidade). Aplicado a IA, isso vira uma política escrita:

Apetite: aceitamos geração de código por LLM em módulos não críticos. **Tolerância:** limite de X% de defeitos escapados atribuíveis a código gerado. Se ultrapassar, **suspende-se a geração assistida em módulos de pagamento.**

Isso é exatamente o formato do exemplo de SRE que ele usa em aula (SLO de 99,9%, e ao passar do limiar o time para de lançar features). **Governança de IA escrita com as palavras dele.**

Uma matriz de riscos de verdade, nas colunas exatas da planilha da Aula 7 (ID | Risco | Categoria | P (1-5) | I (1-5) | Exposição | Tipo de Ação | Responsável | Visibilidade | Plano):

Risco	Categoria	Visibilidade	Resposta
Alucinação de API inexistente	Técnico	Interno	Mitigar — CI quebra no build

Risco	Categoria	Visibilidade	Resposta
Vazamento de dado sensível em prompt	Organizacional	Cliente (LGPD)	Evitar — política de ferramentas aprovadas
Dependência de fornecedor de modelo	Organizacional	Cliente	Mitigar — abstração e contingência
Erosão do conhecimento da equipe	Pessoas	Interno	Mitigar — rodízio de revisão
Propriedade intelectual do código gerado	Organizacional	Cliente (contratual)	Transferir — análise jurídica

A pergunta mais original do seminário inteiro, derivada da **transparência seletiva** (Aula 7): o uso de LLM é risco interno ou compartilhado? Pelo critério do professor, começa como risco **Técnico** (interno), mas migra para **compartilhado** assim que toca propriedade intelectual, LGPD e compliance — categorias que ele lista explicitamente como "devem ser comunicadas ao cliente". Conclusão: **existe um dever de disclosure do uso de IA**, derivado diretamente do critério dele. E, por extensão, a declaração de uso de IA deveria ser item do **Termo de Aceite** (Aula 10), não só do TAP.

O que a própria profissão decidiu. Enquanto o debate público discute se a IA substitui o desenvolvedor, os dois padrões oficiais — PMBOK 8ª ed. (2025) e o **Standard for AI in Portfolio, Program and Project Management** (PMI, jun/2026, 275 páginas, primeiro padrão de IA aprovado pela ANSI para a profissão) — institucionalizaram **human-in-the-loop e governança**. A aposta institucional é conservadora, e ela diz: o profissional é **quem decide sobre recomendações**, não quem opera a ferramenta.

Como se preparar

Seja honesto: a melhor síntese disponível é **opinião especializada**, não evidência. Diga isso. Habilidades que ganham valor, segundo mesas-redondas com indústria e academia em 2026:

1. **Verificação e validação** — avaliar artefatos em vez de escrever código.
2. **Criação de especificação** — converter requisito de negócio em critério verificável por máquina.
3. **Orquestração de agentes.**
4. **Gestão de "dívida cognitiva"** — preservar entendimento arquitetural enquanto a IA gera código.
5. **Expertise de domínio** — fora da engenharia de software pura.

A evidência quantitativa sustenta indiretamente: **35-40% de ganho em greenfield contra ~10% em legado** significa que o valor se desloca para quem entende sistemas existentes. E o mecanismo medido por Stanford é de **entrada**, não de demissão — então a mitigação é ter algo que não seja substituível na entrada.

Demo funcional

Extrapolador da curva METR — a plateia decide o futuro. Gráfico semilog: eixo Y em minutos (log), eixo X em datas. Os pontos medidos já plotados. O público arrasta um slider de **tempo de duplicação** e a reta gira, mostrando em tempo real quando a curva cruza 8 h, 40 h, 167 h e 2000 h.

Os dados exatos (para quem for implementar):

```
// METR Time Horizon 1.1 – horizonte de 50%, em minutos
const metr = [
  { model: "GPT-4 0314",      date: "2023-03-14", p50: 3.5, lo: 1.6, hi:
6.9 },
  { model: "GPT-4o",          date: "2024-05-13", p50: 6,   lo: 2.8, hi:
12.4 },
  { model: "Claude 3.7 Sonnet", date: "2025-02-24", p50: 60,  lo: 32,  hi:
106 },
  { model: "o3",              date: "2025-04-16", p50: 121, lo: 74,  hi:
201 },
  { model: "GPT-5",           date: "2025-08-07", p50: 214, lo: 117, hi:
480 },
  { model: "Claude Opus 4.5",  date: "2025-11-24", p50: 320, lo: 170, hi:
729 },
  { model: "Claude Opus 4.6",  date: "2026-02-20", p50: 870, lo: 360, hi:
5880 }
];
const duplicacao = [196.5, 130.8, 88.6]; // dias – série completa, desde
2023, desde 2024
// cruzamento = dataAncora + log2(alvo / horizonteAncora) × T2
```

Os dois elementos que fazem a demo valer nota:

1. **A linha vermelha tracejada em 16 h**, rotulada *"a partir daqui, a própria METR diz que não sabe medir"*, com toda a região acima sombreada como extrapolação não verificável.
2. **O botão "modo cético"**, que troca as estimativas centrais pelos limites inferiores dos ICs. A data de cruzamento salta anos. **Esse contraste é o argumento inteiro do bloco**, e ele acontece com um clique.

Painel lateral sugerido: "quem ganha dinheiro com esta previsão". Um mapa rápido de conflitos de interesse — OpenAI (GDPval), Google (DORA), Scale AI (SWE-bench Pro), GitClear (relatório de qualidade), METR (sem fins lucrativos). É provavelmente o slide mais educativo da apresentação inteira.

Falas-chave

"Em 2023 o melhor modelo aguentava três minutos e meio de trabalho humano. Em fevereiro de 2026, quatorze horas e meia. E aí a METR escreveu na página: a partir de dezesseis horas, a gente não sabe mais medir."

"A profissão vai crescer quinze por cento na década. E mesmo assim pode não ter vaga para a gente quando sair daqui. Os dois números são oficiais."

Fecho da apresentação inteira — volte para a Aula 1. O professor abriu a disciplina com um caso de **viés algorítmico em sistemas de IA** classificado como falha de engenharia de software, ao lado do Therac-25 e do Ariane 5. O tema nunca foi externo à disciplina. Ele estava na primeira aula.

"Se implementar deixou de ser o gargalo, onde o gerente deve investir? Requisitos, arquitetura, validação, segurança, integração, feedback e governança. Que é, literalmente, a ementa desta disciplina."

Fontes do bloco

[METR — Time Horizon 1.1 \(jan/2026\)](#) · [METR — Task-Completion Time Horizons](#) · [Transformer — Against the METR graph](#) · [Scale AI — SWE-bench Pro leaderboard](#) · [arXiv — The SWE-Bench Illusion](#) · [OpenAI — GDPval](#) · [Stanford — Canaries in the Coal Mine \(ago/2026\)](#) · [BLS — AI, IT and employment 2024-34](#) · [NBER w35046 — Forecasting the Economic Effects of AI](#) · [Metaculus — Date of AGI](#) · [PMI — Standard for AI \(jun/2026\)](#) · [arXiv — Skills for the future software profession](#)

Plano de design e interatividade evolutiva

A ideia do grupo — o slide envelhecer junto com a narrativa — é boa e vale ser levada ao extremo. **A mudança visual não é decoração: ela é o argumento.** O público deve sentir o tempo passando sem que ninguém precise dizer "agora estamos em 2017".

A regra de ouro

Cada bloco termina com uma **demo funcional que o público vê rodar**, e a transição para o próximo bloco **redesenha o slide na frente deles**. Feita com uma transição CSS de 1,5 s sobre variáveis de tema, isso custa quase nada de implementação e vale muito no critério "qualidade do material de apoio".

Bloco	Estética	Tipografia	Paleta	Demo que fecha o bloco
1 · 1948–2016	Terminal CRT, scanlines sutis, cursor piscando	Monoespaçada	Verde fósforo sobre preto	ELIZA com painel de regras
2 · 2017–2022	Web flat dos anos 2010, cartões, sombras leves	Sans-serif geométrica	Azul e branco	Tokenizador + mapa de atenção
3 · Hoje I	IDE escura contemporânea, painel lateral de agente	Mono para código, sans para texto	Escuro com acentos	Agente resolvendo uma issue
4 · Hoje II	Dashboard claro, gráficos vivos — contraste deliberado	Sans neutra	Claro, dados coloridos	Simulador de gargalo
5 · Futuro	Quase vazio: só o gráfico e a plateia decidindo	Fina, muito espaço	Neutra, uma linha vermelha	Extrapolador da curva METR

Como implementar sem sofrimento

Um único arquivo HTML, biblioteca de slides simples (**Reveal.js** serve bem), com um `data-era` em cada seção e variáveis CSS por era:

```
:root { --bg: #000; --fg: #33ff66; --font: ui-monospace, monospace; }
[data-era="2017"] { --bg: #f7f9fc; --fg: #1a4b8c; --font: system-ui, sans-serif; }
body { background: var(--bg); color: var(--fg); font-family: var(--font);
  transition: background .9s ease, color .9s ease; }
```

Cada demo é um componente isolado no mesmo arquivo. **Regra inegociável: nada depende de rede, de API ou de CDN na hora da apresentação.** Tudo inline.

Onde a plateia participa

Quatro momentos, espalhados para não cansar:

1. **Bloco 1** — peça uma frase da plateia para responder no ELIZA. Depois mostre a regra que gerou a resposta.
2. **Bloco 2** — peça uma palavra e uma linha de código para tokenizar lado a lado.
3. **Bloco 4 — o melhor momento da apresentação:** convide alguém a arrastar o slider do simulador de gargalo e pergunte antes *"o que vocês acham que vai acontecer com o prazo de entrega?"*. A plateia vai prever que cai. Não cai. A surpresa é o argumento.
4. **Bloco 5** — peça um palpite de ano para "IA faz uma semana de trabalho humano" antes de revelar a curva. Depois aperte o **modo cético** na frente deles.

Ideias extras, se sobrar fôlego

- **"Humano ou IA?"** — três trechos de código, plateia vota levantando a mão. Um deles tem uma vulnerabilidade sutil. Serve de ponte para a seção de riscos.
- **Relógio de era no canto do slide** — um contador discreto (1948 → 2026 → ?) que avança nas transições. Custa 10 linhas e amarra a metáfora da linha do tempo.
- **QR code na abertura** apontando para os slides publicados, para quem quiser mexer nas demos depois. Também demonstra organização — critério avaliado.

Riscos da apresentação — e as respostas

Ironicamente, o melhor jeito de mostrar domínio de GP é gerenciar o risco da própria apresentação.

Risco	P × I	Resposta
Demo trava ou não carrega no projetor	Alta × Alto	Testar na sala antes; ter um GIF de fallback de cada demo no próprio slide
Estourar o tempo	Alta × Médio	Marcar em cada bloco o parágrafo cortável; cronometrar no ensaio
Plateia não interage	Média × Baixo	Combinar antes com dois colegas para puxar a primeira participação
Pergunta técnica fora do roteiro	Média × Médio	Seção de perguntas prováveis abaixo, decorada
Notebook do apresentador falha	Baixa × Alto	O HTML em pendrive e publicado num link

Cronometragem, divisão e plano de ensaio

Alvo: **23 minutos**, dentro da janela de 20–25 com folga para a demo travar e ser recuperada.

#	Bloco	Apresentador	Tempo	Corte de emergência se atrasar
—	Abertura e tese	Apresentador 1	1 min	Não cortar
1	1948–2016 — as origens	[DECIDIR]	4 min	Cortar o gerador n-grama, manter ELIZA
2	2017–2022 — o Transformer	[DECIDIR]	3 min	Cortar o mapa de atenção, manter a tabela LLM/assistente/agente
3	Hoje I — o ciclo de vida e o caso GitHub	[DECIDIR]	5 min	Cortar Brooks e PNR, manter a EAP
4	Hoje II — a evidência e o gargalo	[DECIDIR]	5 min	Cortar o burndown, manter o simulador
5	Futuro — a curva e os cenários	[DECIDIR]	4 min	Cortar a tabela de cenários, manter a curva e o modo cético
—	Fecho e volta à Aula 1	Apresentador 5	1 min	Não cortar

[DECIDIR] Quem fica com qual bloco. Uma sugestão: o bloco 4 é o mais denso em números e o que mais decide a nota de profundidade — dê a quem tiver mais confortável com dados. O bloco 1 é o mais narrativo e serve bem a quem prefere contar história.

Plano de ensaio

Três ensaios, cada um com um objetivo diferente. Não é exagero — são cinco pessoas e cinco transições, e transição mal ensaiada é o que mais custa ponto em "comunicação, postura e gestão do tempo".

- 1. Ensaio de conteúdo** (sem slide) — cada um conta seu bloco em voz alta a partir deste documento. Objetivo: descobrir o que ninguém consegue explicar sem ler. O que não sair naturalmente, sai do roteiro.
- 2. Ensaio de transição** (com slide) — ensaiar **só os 30 segundos de virada entre blocos**, umas cinco vezes seguidas. É aí que apresentação em grupo quebra.
- 3. Ensaio cronometrado completo**, na sala real, com o projetor real. Um colega fora do grupo marca o tempo por bloco e anota onde perdeu atenção.

Checklist do dia

- ☐ HTML funcionando offline, testado no navegador do notebook que vai ser usado
- ☐ GIF de fallback de cada demo embutido no próprio slide
- ☐ Pendrive com o arquivo e link publicado
- ☐ Um cronômetro visível para quem está falando
- ☐ Dois colegas combinados para puxar a primeira interação da plateia
- ☐ Seção de perguntas prováveis lida por todos, não só pelo dono do bloco

Perguntas prováveis — com resposta preparada

Todos do grupo devem conhecer estas respostas, não só o dono do bloco. Respostas curtas e com número.

"Então a IA aumenta a produtividade ou não?" Em muitos contextos sim, universalmente não. O maior experimento corporativo achou +26,08% em tarefas concluídas com 4.867 desenvolvedores. Um RCT com 16 mantenedores experientes achou 19% mais lentos — e os próprios autores revisaram esse desenho em fev/2026 por viés de seleção. O DORA 2026 dá o mapa: 35–40% em greenfield, ~10% ou menos em legado.

"LLMs vão substituir desenvolvedores?" Os estudos sustentam automação de tarefas, não substituição da profissão. O BLS projeta +15,8% de vagas na década. Mas a evidência de Stanford mostra a **porta de entrada** apertando: 22–25 anos em ocupações expostas estão ~19% abaixo dos pares, por redução de contratação. O risco é de entrada, não de extinção.

"Mas o GitHub não fez 800 mil linhas com uma pessoa? Isso não prova substituição?" Prova alavancagem enorme de um profissional, num caso de fornecedor, relatado por ele mesmo. E o próprio relato deixa claro que o que tornou aquilo possível foi a **disciplina de engenharia**: 128 PRs, main sempre entregável, E2E a cada passo, pilotos antes da escala. Sem isso, o mesmo volume de código seria um desastre.

"Qual é o papel do gerente de projetos nesse cenário?" Selecionar onde a IA gera valor, definir política e guardrails, refazer as estimativas quando o baseline fica obsoleto, garantir quality gates, medir o fluxo inteiro e impedir que produtividade local seja confundida com desempenho do projeto. É o que o PMBOK 8 e o padrão de IA do PMI (2026) formalizaram.

"Por que não medir quantas linhas a IA produziu?" Porque é output local, não outcome. E porque é Goodhart: ao virar meta, a métrica se destrói — o time aceita sugestões ruins para bater o indicador. Nos critérios da Aula 5, a métrica é prática e confiável, mas **não é válida**.

"Código de IA é confiável?" Não por padrão — é artefato a ser verificado, como qualquer outro. Um estudo empírico com 733 trechos atribuídos a assistentes achou weaknesses de segurança em 29,5% dos exemplos Python e 24,2% dos JavaScript, em 43 categorias CWE. Contextualize: números daquelas ferramentas e amostra, não taxa universal em 2026. O que importa é o princípio: **funcionar não é o mesmo que ser seguro**.

"Vocês usaram IA para fazer esta apresentação?" Pergunta provável, e a resposta honesta é a melhor: sim, para pesquisa e organização do material, com verificação humana das fontes — que é exatamente o modelo *human-in-the-loop* defendido no bloco 5. Se alguém do grupo não estiver confortável com essa resposta, combinem antes.

"Esses números da METR não são muito otimistas?" São discutíveis, e nós mostramos por quê: nas tarefas mais realistas nenhum modelo passa de 30%; ~34% das tarefas têm solução pública no treino; só 5 das 31 tarefas longas têm tempo humano realmente medido; e o IC do Opus 4.6 vai de 6 a 98 horas. Por isso a demo tem modo cético.

Fontes

As 8 mais importantes estão em negrito — se o professor perguntar "em que vocês se basearam", são essas.

História (bloco 1)

[Turing — Computing Machinery and Intelligence \(1950\)](#) · [Shannon — A Mathematical Theory of Communication \(1948\)](#) · [Shannon — Prediction and Entropy of Printed English \(1951\)](#) · [Hutchins — The Georgetown-IBM Experiment \(2004\)](#) · [Hutchins — ALPAC, the \(in\)famous report \(1996\)](#) · [Weizenbaum — ELIZA, CACM \(1966\)](#) · [ELIZA Reanimated — código original reencontrado \(2025\)](#) · **[Hindle et al. — On the Naturalness of Software, ICSE 2012](#)** · [Brown et al. — Statistical Machine Translation \(1993\)](#) · [Karpathy — The Unreasonable Effectiveness of RNNs \(2015\)](#)

Técnica (bloco 2)

[Vaswani et al. — Attention Is All You Need \(2017\)](#) · [Bahdanau et al. — Neural MT by Jointly Learning to Align and Translate \(2014\)](#) · [Jimenez et al. — SWE-bench, ICLR 2024](#)

Prática e casos (bloco 3)

[GitHub — Migrating the Copilot runtime to Rust, using Copilot \(16/09/2026\)](#) · [Fu et al. — Security Weaknesses of Copilot-Generated Code](#)

Evidência empírica (bloco 4) — o núcleo da pesquisa

[Cui et al. — The Effects of Generative AI on High-Skilled Work, Management Science \(2026\)](#) · [METR — RCT com desenvolvedores experientes \(jul/2025\)](#) · [METR — mudança do desenho do experimento \(fev/2026\)](#) · [DORA — State of AI-assisted Software Development \(2025\)](#) · [DORA — ROI of AI-assisted Software Development \(2026\)](#) · [GitClear — The Maintainability Gap \(2026\)](#) · [Stack Overflow Developer Survey 2025](#)

Futuro (bloco 5)

[METR — Time Horizon 1.1 \(jan/2026\)](#) · [METR — Task-Completion Time Horizons](#) · [Transformer — Against the METR graph](#) · [Scale AI — SWE-bench Pro](#) · [The SWE-Bench Illusion](#) · [OpenAI — GDPval](#) · [Stanford Digital Economy Lab — Canaries in the Coal Mine \(ago/2026\)](#) · [BLS — AI, IT and employment 2024–34](#) · [NBER w35046 — Forecasting the Economic Effects of AI](#) · [Metaculus — Date of AGI](#) · [Epoch AI — Can AI scaling continue through 2030?](#) · [Skills for the future software profession \(2026\)](#)

Gerenciamento de Projetos

[PMI — The Standard for AI in Portfolio, Program and Project Management \(jun/2026\)](#) · [PMI — PMBOK Guide, 8ª edição \(2025\)](#) · Slides das Aulas 1 a 10 do prof. Bruno Elias Penteado, mais as planilhas de matriz de riscos (Aula 7), critérios de qualidade (Aula 9) e a atividade EAV × Burndown (Aula 10) — todos no Projeto **GP-trabalho**.

Material interno do grupo

Deep research prévio no projeto (`deep-research-report.md` e o PDF equivalente) · TAP, MVV/OKR, Team Charter Híbrido, KLOC e orçamento do UNESPÃO · repositório [00davidсарago00/unespao](#)

Nota sobre verificação

Alguns pontos ficaram com confirmação parcial e **não devem ser afirmados como fato**: as datas exatas de lançamento dos modelos na tabela da METR; a trajetória ano a ano do SWE-bench Verified (2023 ~4%, 2024 ~49%) — amplamente citada, não confirmada em fonte primária; o orçamento exato do projeto japonês Fifth Generation (fontes divergem); e a redação exata da frase atribuída a Jelinek sobre demitir linguístas — ele próprio, em 2004, disse esperar que fosse apócrifa mas admitiu ter dito algo parecido.